

Haithem Farjallah

Berlin, Allemagne | +49 155 1106 3831

haithem.farjallah.software@gmail.com | linkedin.com/in/haithemfarjallah | github.com/haithemfarjallah

Formation

Faculté des Sciences de Tunis, Université de Tunis El Manar
Diplôme National d'Ingénieur en Informatique

Tunis, Tunisie
2021 – 2026

Compétences

Langages : Java, TypeScript, JavaScript, Python, C, C++

Technologies : Spring Boot, Node.js, PostgreSQL, MongoDB, Redis, Apache Kafka, React, Angular

DevOps : AWS, Docker, Kubernetes, Terraform, Helm, ArgoCD, GitHub Actions, Jenkins, Prometheus, Grafana

Outils : Git, Bash, Linux, Maven, JUnit

Expérience professionnelle

Projet de fin d'études, Alta Via Consulting – Berlin, Allemagne

Avril 2026 – Septembre 2026

- Création d'une solution de synchronisation bidirectionnelle entre NetSuite et le logiciel de comptabilité allemand DATEV, couvrant les clients, les fournisseurs et les employés à l'aide de Node.js et SuiteScript remplaçant les exports manuels de fichiers CSV que les comptables effectuaient entre les 2 systèmes
- Création d'un tableau de bord React affichant côte à côte les enregistrements DATEV et NetSuite, avec leur statut et les mises à jour en attente, permettant aux comptables de valider des champs individuellement
- Intégration d'AWS Lambda pour la transformation des données DATEV en enregistrements NetSuite, incluant le mapping de plus de 10 champs avec conversion de types et validation
- Implémentation d'un pipeline CI/CD qui exécute des tests unitaires et déploie les mises à jour dans les comptes de production, remplaçant les installations manuelles et réduisant le temps de déploiement de 30 à 6 minutes

Développeur full stack, Dipawer – Tunis, Tunisie

Août 2025 – Mars 2026

- Développement d'une plateforme vétérinaire basée sur une architecture microservices, couvrant la prise de rendez-vous, les consultations vétérinaires et les paiements en ligne, avec Spring Boot, Angular et PostgreSQL
- Implémentation du module de gestion de la santé permettant le stockage des dossiers médicaux, l'historique des vaccinations et les rappels programmés afin d'aider les propriétaires à suivre l'état de leurs animaux
- Ajout d'un workflow de gestion des urgences, reliant les propriétaires d'animaux à plus de 10 cliniques partenaires et intégrant l'assistance vétérinaire, la disponibilité des cliniques, le routage géolocalisé et les demandes d'ambulance pour les cas urgents

Projets

Sandbox MCP – Spring Boot, Docker, MCP Java SDK, GitHub Actions

- Création d'un serveur MCP permettant aux LLM d'exécuter et de tester du code généré dans des conteneurs sécurisés afin de le valider et le corriger sans exécuter directement de code non fiable sur la machine hôte
- Renforcement de l'isolation à l'exécution avec des limites de CPU, de mémoire et de PID, système de fichiers en lecture seule et utilisateur non root avec nettoyage automatique des conteneurs
- Conception d'environnements d'exécution configurables pour différents langages à l'aide de Dockerfiles et d'une configuration JSON, permettant d'ajouter de nouveaux langages sans recompiler le serveur

Plateforme de programmation compétitive – Spring Boot, React, PostgreSQL, AWS

- Développement d'une plateforme de programmation compétitive 1v1 avec matchmaking, chronomètres de match synchronisés, résultats de soumission en temps réel et communication basée sur WebSocket
- Mise en place d'une architecture événementielle avec un backend principal et un service Sentinel dédié, utilisant AWS SQS pour traiter de manière asynchrone les soumissions de code et les appels à l'API Codeforces
- Migration du déploiement manuel sur AWS vers une infrastructure Terraform, avec déploiement des services backend sur ECS Fargate derrière un Load Balancer et hébergement du frontend React sur S3 avec CloudFront

Application de réseau social – Spring Boot, Kafka, Kubernetes, Helm, ArgoCD, Jenkins

- Mise en place de 5 microservices Spring Boot communiquant via REST et Kafka, chacun avec sa propre base de données PostgreSQL, permettant de les déployer et de les mettre à l'échelle indépendamment
- Automatisation d'un pipeline CI/CD pour exécuter les tests, construire les images Docker et les publier sur Azure Container Registry, avec synchronisation des modifications vers Azure Kubernetes Service via ArgoCD
- Déploiement des microservices sur Kubernetes à l'aide de charts Helm versionnés, permettant de gérer indépendamment les versions, la configuration et le rollback de chaque service